Lab Tasks

Task # 01:

A transportation company manages different types of vehicles. All vehicles share some common properties like vehicle ID, brand, speed, and fuel type. These common features are defined in a base class Vehicle. Different types of vehicles inherit from this base class and add their own specific behaviors.

Base Class:

- **Vehicle**

- Properties: vehicleID, brand, speed, fuelType.
- Method start() prints that the vehicle has started.
- Method stop() prints that the vehicle has stopped.
- Method displayInfo() prints the vehicle's ID, brand, speed, and fuel type.

Child Classes:

1. **Car** (inherits Vehicle)
    - Method openSunroof() prints "Sunroof opened."
    - Method activateCruiseControl() prints "Cruise control activated."
2. **Truck** (inherits Vehicle)
    - Method loadCargo(int weight) prints the cargo weight loaded.
    - Method unloadCargo() prints "Cargo unloaded."
3. **Motorbike** (inherits Vehicle)
    - Method popWheelie() prints "Popping a wheelie!"
    - Method enableABS() prints "ABS enabled."
4. **Bus** (inherits Vehicle)
    - Method pickPassengers(int count) prints number of passengers picked.
    - Method dropPassengers() prints "Passengers dropped."

Task:

- Design a base class Vehicle with common properties and methods.
- Create the four child classes (Car, Truck, Motorbike, Bus) that inherit from Vehicle and implement their own specific methods.
- Demonstrate hierarchical inheritance by showing that all vehicles share base class features but also have unique behaviors.
- Write a main program to create objects of all vehicle types, call both common methods (start, displayInfo) and vehicle-specific methods (openSunroof, loadCargo, popWheelie, pickPassengers), and print results to verify inheritance works properly.

Task # 02:

A company wants to develop a smart office system where devices can have multiple capabilities. Each device can have communication capability and power management capability. In Java, multiple inheritance can be simulated using interfaces.

Interfaces:

1. **Communication** – method connect(String network) to connect to Wi-Fi or LAN.
2. **PowerManagement** – method powerOn() to switch on, powerOff() to switch off, and checkBattery() to return battery level.

Device Classes:

- **Printer** implements Communication and PowerManagement.
    - connect(String network) prints “Printer connected to network.”
    - powerOn() prints “Printer powered on.”
    - checkBattery() prints battery status.
- **Scanner** implements Communication and PowerManagement.
    - connect(String network) prints “Scanner connected to network.”
    - powerOn() prints “Scanner powered on.”
    - checkBattery() prints battery status.

Task:

- Design interfaces Communication and PowerManagement.
- Create Printer and Scanner classes that implement both interfaces.
- Write a main program to create objects of Printer and Scanner, call all methods from both interfaces, and show that both capabilities are available for each device.

Example Scenario:

- Connect the printer and scanner to the office Wi-Fi.
- Power on both devices and check their battery status.

Use interfaces to let a single class have different capabilities from multiple sources, demonstrating that one class can perform more than one type of action.

Task # 03:

A university wants to manage different types of staff with shared and unique behaviors. The system uses interfaces and a base class to simulate hybrid inheritance.

Base Class:

- Staff – properties name, id, department.
- Method displayInfo() prints staff information.

Interfaces:

1. **Teaching** – method conductLecture() prints "Conducting lecture."
2. **Research** – method publishPaper() prints "Publishing research paper."

Child Classes:

- **Professor** – inherits Staff, implements Teaching and Research.
    - Can conduct lectures and publish papers**.**
- **LabInstructor** – inherits Staff, implements Teaching.
    - Can conduct lectures but not publish papers.

- **ResearchAssistant** – inherits Staff, implements Research.
    - Can publish papers but not conduct lectures.

Task:

- Design Staff class and interfaces Teaching and Research.
- Create Professor, LabInstructor, and ResearchAssistant classes using hybrid inheritance.
- Write a main program to create objects of each staff type, call all relevant methods (lecture, publish), and print results.
- Include conditions: e.g., if a ResearchAssistant tries to conduct a lecture, the program should ignore or display "Not allowed."

Example Scenario:

- A Professor conducts a lecture and publishes a paper.
- A LabInstructor conducts a lecture only.
- A ResearchAssistant publishes a paper only.

Use hybrid inheritance, combining class inheritance and interfaces, allowing multiple types of behavior in different subclasses.